FormPilot: Complete Project Guide

Workflow, files, terms, how to run, how to show the tests, and likely judge questions

1. What the project is, in one minute

Many Indian government and exam websites are messy: popups, dead links, OTPs, CAPTCHAs, sessions that expire. **FormPilot is an AI agent that fills such a website the way a careful person would: it looks at the screen, decides one action, does it, checks what happened, and repeats.** The AI model runs fully on your own Mac, so screenshots never leave your machine and no paid API is needed.

The special part (Challenge 1, “The click you can't undo”) is what happens around **payments**. A payment cannot be taken back, so the agent treats it with extra care: it asks you first, it never pays twice, and if it crashes or the session expires it recovers without paying again. After feedback on the Reddit post it also has a **hard spending limit**, a **dry-run mode** that stops right before the payment, and approval for **any** irreversible button, not only payments.

Simple picture: the AI model is the **brain** (looks at a picture, says what to do). Playwright is the **hands** (really clicks and types in a browser). Python code in between is the **safety officer** (checks every instruction, blocks dangerous ones, keeps a diary).

2. The workflow, step by step

Every step of a run repeats this loop (all in `agent/agent.py`):

#	Stage	What happens in plain words
1	Look	Take a screenshot of the browser.
2	Check the world	If a payment was attempted earlier, look at the page text for proof of what happened (receipt, “Payment received”).
3	Ask the brain	Send the screenshot, the task, your profile, recent history and warnings to the local model. It answers with ONE action as JSON (click here, type this, pick that).
4	Validate	Strictly check the answer. Wrong format, coordinates outside the screen, odd characters: rejected and the model is asked again (a few tries only).
5	Guard	If the model tries to type its own OTP or CAPTCHA, block it and ask the human instead.
6	Classify	Look at what is really under the pointer on the page and decide the action's risk class (see section 5).
7	Hold if incomplete	If the click would submit a form that still has required fields empty, do not click; tell the model what is missing.
8	Spending limit	For a payment: read the amount from the page. Over the limit (or unreadable): no click, no approval screen, the run stops.
9	Dry run?	If dry run is on: stop here, report what would have been sent, send nothing.
10	Ask the human	For any irreversible click, show an approval screen with the amount and what will be sent. Deny stops everything.
11	Log intent	Write “about to do this” to disk **before** clicking.
12	Act	Playwright performs the click, typing, or dropdown choice.
13	Verify	Did the page change? Does the typed text really sit in the field? For a payment: is there proof it went through?
14	Record	Write the step to the trace and save a checkpoint so the run can resume after a crash.

The loop ends when the page shows the real receipt (the agent cannot just claim “done”), when the step limit (60) is reached, when the human presses Stop or denies something, when a payment is over the spending limit, when a dry run reaches the irreversible click, or when the agent is stuck and gives up.

3. The mock portal (the website being automated)

`portal/app.py` is a fake exam-registration site built to look like a real messy one. It is a mock: no real government site is touched and no real money moves. The journey is:

landing page with popups → register → personal details → OTP → exam centre → CAPTCHA → review → fee payment → receipt

It also contains traps and test hooks: a **Pay Again** button, a session that can **expire right after payment**, an SMS inbox page (`/__sms`) that plays the role of your phone, and a ledger page (`/__ledger`) that lists every charge it took. That ledger is the **ground truth** for proving there was only one payment: it is the portal's own record, not the agent's opinion of itself.

4. Every file and what it does

File	Lines	What it is, in simple words
agent/agent.py	797	The main loop (section 2). Also: pre-submit check, spending limit, dry run, approval screen, OTP/CAPTCHA guard, ledger handling, checkpoints, resume, trace writing, loop detection.
agent/vision.py	200	Talks to the local AI model through Ollama. Builds the prompt, sends the screenshot, asks for strict JSON, retries a limited number of times, reports if the model is missing or Ollama is off.
agent/actions.py	289	The only actions the model may produce: click, type, select, press, scroll, wait, screenshot, ask_human, done, error. Validates them and runs them in the browser. No shell, no arbitrary code.
agent/consequence.py	245	The risk policy: action classes, the DOM probe (what is really under the pointer), the built-in list of irreversible button labels, reading rupee amounts, commit specs, and the write-ahead ledger (the diary of irreversible actions).
agent/config.py	109	Settings from the .env file and the task file format.
prompts/system.md	25	The instructions given to the model every step: the allowed actions and the rules (dismiss popups, never invent data, ask for OTP/CAPTCHA, never pay twice...).
tasks/sbee_exam.json	43	The task: the goal, the fake user profile (name, date of birth...), hints for the model, the success pattern, which values are secret, which button is a payment, and the spending limit for it (max_amount: 1000).
portal/app.py	422	The fake exam website (section 3).
server.py	174	Backend of the dashboard (FastAPI). Starts runs, streams live events to the page, receives your answers (OTP, approve, deny).
ui/index.html	122	The dashboard page: live browser view, action trace, list of irreversible actions, the approval screen, dry-run tick box, spending-limit box, failure-injection menu, Run / Resume / Stop.
setup.sh	106	One command for a new machine: installs Ollama if missing, creates the Python environment, installs packages and Chromium, downloads the model, then starts everything.
run.sh	31	One command to start the portal (port 5050) and dashboard (port 8000), after checking Ollama and the model.
scripts/demo_challenge.py	85	The challenge proof: runs the happy path and three failure scenarios, then checks the portal's ledger shows exactly one charge each.
scripts/benchmark.py	65	Runs the same task several times per model and reports success rate and time.
scripts/export_video.py	25	Turns a run's screenshots into an MP4 (needs ffmpeg).
tests/	~830	84 automated tests (section 8). helpers.py has the scripted stand-in model and a pretend human; conftest.py starts a test portal.

File	Lines	What it is, in simple words
FormPilot_Demo.mp4	-	The demo video: 12 test cases, each with its problem, how it is handled, and the result.
screenshots/	-	The Reddit post, the feedback comment, and four dashboard screenshots of the changes made in response (dry run, limit block, approval with limit, paid once).
FormPilot_Project_Guide.pdf	-	This guide.
README.md, .env.example, requirements.txt, pytest.ini, .gitignore	-	Documentation, example settings, Python packages, test settings, files kept out of Git (.env, runs/, .venv/).

5. Technical terms explained simply

Term	Meaning
Agent	A program that repeatedly looks at a situation, decides what to do, does it, and checks the result.
VLM (vision-language model)	An AI that understands pictures and text together. Here: Qwen3-VL 4B. You show it a screenshot and it says where to click.
Ollama	A free app that runs AI models on your own computer. FormPilot talks to it at localhost:11434.
Local / on-device	Everything runs on your Mac. Screenshots are never sent to the internet.
Playwright	A tool that controls a real Chrome browser from code: open pages, click at x,y, type, take screenshots.
Headless	The browser runs without showing a window.
Screenshot-based (vision-first)	The model decides only from the picture, like a person would. The page's hidden code (DOM) is used only to *check* results, never to choose what to click.
DOM	The page's internal structure (buttons, fields). Used here as *evidence*.
Coordinates / norm1000	Where to click as x,y. Qwen3-VL uses a 0 to 1000 grid regardless of screen size; the code converts it to real pixels.
JSON / structured output	A strict text format for the model's answer, e.g. {"action":"click","x":500,"y":300}. Ollama forces the model to follow a schema.
Schema validation (pydantic)	Automatic checking that an answer has the right fields, types and limits before anything is done.
Restricted executor	The part that runs actions. It only knows 10 safe action types: no shell, no scripts, no file access.
Reversible vs irreversible	Typing in a box can be redone. A payment cannot be undone.
Action classes	observe (just look), local (edit this page), navigate (go to the next step), commit (irreversible).
Commit / commit spec	An irreversible action, and the rule that recognises it (button text like “Pay Now”, plus what proves it worked or failed).
DOM probe	A quick check of what element is actually under the click point. The real button label beats what the model *says* it is clicking.
Ledger / write-ahead log	A diary of payments, written to disk *before* the click. Statuses: intent → dispatched → confirmed / rejected / in_doubt.
in_doubt	The click was sent but the page shows neither success nor failure (timeout, expired session). Treated as *maybe paid*, so a second payment is blocked.
Idempotency / no duplicate	Doing something twice has the same effect as once. Here: never charging twice.

Term	Meaning
Reconciliation	Comparing what the agent believes with proof on the page (receipt, “Payment received”) and updating the ledger.
Checkpoint / resume	After every step the run's state and browser login are saved, so a crashed run continues from where it stood instead of restarting (no second OTP, CAPTCHA or payment).
Fault injection	Breaking things on purpose to prove recovery: session expiry after payment, or killing the agent right after the Pay click.
Human-in-the-loop	The agent stops and asks you for things only you should do: OTP, CAPTCHA, approval to pay.
Human-first approval	Before an irreversible click, a screen shows the amount and what will be sent. Approve or Deny.
Spending limit (hard cap)	A maximum amount in the task file (max_amount). The agent reads the amount from the page; above the limit, or if no amount can be read, the payment click is never made and it is not even offered for approval.
Dry run	A rehearsal: the agent does everything for real up to the irreversible click, then stops and reports what it would have sent. Nothing is charged. Resume continues the same run for real.
Irreversible labels	A built-in list of button texts that cannot be undone on any site (final submit, delete, cancel application, withdraw, pay...). Such a button is treated like a payment even if the task file and the AI say nothing.
Pre-submit check	Before a submit click, verify the form has no required field empty and no radio group unselected (and, before an irreversible click, no empty box at all). If something is missing, the click is held.
Trace (honest trace)	The step-by-step record: what the model *claimed* to click, what was *really* under the pointer, whether it worked. Marks typed fields verified or mismatch.
Redaction / masking	Secrets (mobile, Aadhaar, UPI ID, OTP) appear as ••• in logs and screens.
Hallucination	When an AI confidently says something wrong. Defence here: never trust its claims; check the page (e.g. “done” is accepted only if the receipt text is on screen).
Mock	A fake copy built for safe testing.
Scripted planner	A fixed, non-AI stand-in for the model used in tests to prove the safety machinery. It is **not** a model result.
Mock vs real model run	Scripted = fast and deterministic, tests the safety logic. Real model = slower, proves the AI can actually navigate.

6. How to run the project

Easiest: one command on a new machine (installs what is missing, downloads the model once, starts everything):

```
git clone https://github.com/dhanushshekhar13-alt/formpilot && cd formpilot
./setup.sh
```

Or by hand (needs Homebrew and Python 3):

```
brew install ollama
ollama serve              # or just open the Ollama app
ollama pull qwen3-vl:4b-instruct   # several GB, one time
cd ~/formpilot
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
playwright install chromium
cp .env.example .env
```

Start the dashboard:

```
./run.sh
```

Open **http://localhost:8000**, keep the task SBEE-2026, leave “Ask me before any irreversible action” ticked, and press **Run agent**. When asked: the OTP appears in the “Your phone (SMS)” box, the CAPTCHA appears as an image, and before payment you get the approval screen. **Stop** is the emergency stop. The fake portal itself is at http://localhost:5050.

Inject a failure: pick one in the dropdown before pressing Run. After a crash a **Resume** button appears.

Handy settings (in `.env`): FORMPILOT_MODEL, FORMPILOT_MAX_STEPS (60), FORMPILOT_HEADED=1 to watch the real browser window, FORMPILOT_APPROVE_COMMITS, FORMPILOT_PRESUBMIT_CHECK, FORMPILOT_DRY_RUN, FORMPILOT_MAX_AMOUNT.

Dashboard options: “Ask me before any irreversible action” (on by default), “Dry run” (stop right before the irreversible click), and “Spending limit Rs.” (a limit for this run; the task file's own limit of Rs. 1000 always applies too).

If something goes wrong: “Ollama is not running” → open the app or `ollama serve`. First step very slow → the model is loading (warm it: `ollama run qwen3-vl:4b-instruct hi`). Port busy → `lsof -iTCP:8000 -sTCP:LISTEN` and kill that process.

7. How to demo the challenge to the judges

Recommended order (about 5 minutes):

- **Show the problem:** one sentence: “a payment can't be undone, and sessions expire or programs crash at the worst moment.”
- **Live happy path** in the dashboard with the real model: show the OTP/CAPTCHA asking you, then the **approval screen**, then success and the portal's ledger showing 1 charge.
- **Show a failure:** choose “session expires right after the payment is taken”. Point at the “Pay Again” being **BLOCKED** in the trace and the ledger going intent → dispatched → in_doubt → confirmed. Then “crash after clicking Pay” and press **Resume**.
- **Show the feedback loop:** open the Reddit comment asking for a spending limit, then tick **Dry run** and run: it stops at Pay Now with nothing sent. Type 500 in the spending-limit box and press **Resume**: it refuses (“Rs. 800 + 18 = 818, over the limit of Rs. 500”). Clear the box, Resume, approve: one payment.
- **Show proof:** run the scripted demo command below, which prints a table of all four scenarios with exactly one charge each.
- **Backup:** the real model is not 100% predictable. If a live run gets stuck, switch straight to the scripted demo and say so honestly.

```
python scripts/demo_challenge.py                    # fast, about 1 minute, scripted driver
python scripts/demo_challenge.py --planner ollama   # real model, about 35 minutes, all 4 scenarios
python scripts/demo_challenge.py --planner ollama --only happy   # one scenario with the real model
```

A ready-made video of all 12 test cases is in the repo: `FormPilot_Demo.mp4`.

Expected end of the scripted output:

```
| scenario                              | status  | portal charges | duplicates blocked |
| happy path                            | success | 1              | 0                  |
| session expires right after payment   | success | 1              | 1                  |
| agent crashes right after Pay click   | success | 1              | 1                  |
| crash + expired session               | success | 1              | 1                  |
ALL SCENARIOS: success with exactly one charge
```

8. How to show all the test cases

There are **84 automated tests**. They need no AI model and take about 4 minutes (they open a real headless browser against a test copy of the portal).

```
.venv/bin/python -m pytest -q          # all 84, short output
.venv/bin/python -m pytest -v          # names of every test with PASSED
.venv/bin/python -m pytest -v tests/test_consequence.py   # just the challenge tests
.venv/bin/python -m pytest -k duplicate -v                # tests whose name contains a word
```

The headline challenge tests live in `tests/test_consequence.py`. In simple words:

File (tests)	What it proves
test_consequence.py (19 test functions)	Happy path pays exactly once and the trace is honest. Session expiry after payment recovers without paying twice. A persistent “Pay Again” is escalated to the human, not executed. Crash after the Pay click then resume. Crash plus expiry together. Incomplete payment form is held before any intent is logged. Human approval shows facts read from the page (UPI ID masked). Declined payment is never sent. A provably rejected payment may be retried. Classification uses what is under the pointer. Ledger blocks repeats until resolved. Guessed OTP/CAPTCHA are replaced by the human's. **Added after feedback:** a payment over the spending limit is never sent; a run-wide limit applies on top of the task's; a capped payment is refused when no amount can be read; dry run stops before Pay and then resumes for real; any irreversible button (Final Submit) needs approval; a denied one is never clicked; an irreversible click is held for any empty box.
test_agent.py (9 test functions)	Full registration reaches a verified receipt with human checkpoints. Step limit ends as incomplete. Repeated no-effect actions escalate then stop. “Done” without a receipt is rejected. Irreversible click needs confirmation and denial stops. Emergency stop works while waiting for a human. Repeated invalid model output ends with a clear message. Model unavailable fails cleanly before the browser opens. The default pipeline does not call any cloud AI.
test_actions.py (15 test functions)	The action validator: bad coordinates, non-numbers, unknown actions, unsafe text, wrong formats are rejected; norm1000 converts to pixels; secrets are masked in logs; executor refuses non-executable actions; dropdown and typing edge cases.
test_vision.py (6 test functions)	The model client: correct request format, recovers from malformed replies, retries are bounded, server-down and model-missing give clear errors.

Tip for the demo: run `pytest -v tests/test_consequence.py` on screen; the test names read like a list of the promises the project makes.

9. Questions judges are likely to ask, with honest answers

What problem does this solve, and for whom?

Filling long, messy government and exam forms is slow and error-prone, and one wrong click on payment costs money. It helps people who find these portals hard, and it shows how to make an AI agent safe around actions that cannot be undone.

Why a local model instead of ChatGPT or Claude?

Privacy: screenshots contain Aadhaar, mobile and payment details, and they never leave the machine. It also costs nothing per run and works offline. The trade-off is that a 4B model is less reliable and slower than a big cloud model.

How do you stop it from paying twice?

Three layers. (1) The payment is written to a diary on disk before the click. (2) While the outcome is unknown (in_doubt) or already confirmed, any repeat of that action is blocked in code, whatever the model says. (3) After a crash it resumes from the checkpoint instead of replaying steps. If it still cannot tell, it asks the human.

What does “in doubt” mean and why is it important?

If a click was sent but we saw no success or failure (timeout, session expired), we cannot assume it failed, so we treat it as possibly paid and block a repeat. Assuming failure is how double charges happen.

What exactly did you inject, and how do you prove one payment?

Two faults at the worst moment: the portal takes the money and then kills the session (and shows a Pay Again trap); and the agent process dies right after the Pay click. The proof is the portal's own charge ledger (/__ledger), which counts exactly one charge in every scenario. We do not trust the agent's own report.

What decides which actions are safe, risky or need a human?

Four classes: observe and local edits are safe to repeat; navigate steps are redone only after re-reading the screen; commits happen exactly once. The class is decided from the real element under the pointer (DOM probe) and the commit rules in the task file, not from what the model claims. Anything irreversible needs the human's approval first.

How is the trace honest about “done versus only appeared to happen”?

Each step records what the model *claimed* to click and what was *really* there, whether the page changed (changed / no_effect), whether typed text actually sits in the field (verified / mismatch), and for payments the status confirmed / rejected / in_doubt / blocked. A “done” is accepted only if the receipt text is on the page.

What if the AI model makes things up or clicks the wrong thing?

Its output is strictly validated, limited to 10 safe actions, and checked against the page afterwards. It cannot run code. For OTP and CAPTCHA the code, not the prompt, blocks a guess. Irreversible actions need human approval. A wrong click on a safe field is simply corrected on a later step.

How does it ask for OTP and CAPTCHA without cheating?

It never solves them. If the model tries to type into an OTP or CAPTCHA field, the code refuses, asks you, and types only your answer. The OTP shows in a simulated phone box in the dashboard.

Is it real or just scripted for your demo?

Both exist and are labelled. The scripted driver proves the safety logic deterministically and is used in tests. The real run uses the local model reading screenshots, and all four scenarios finished with one charge. The model is a small 4B model, so it is not 100% predictable: it sometimes gets stuck on dropdowns, and a run takes about 28 to 50 steps.

What are the limitations?

Tested only on our mock portal, not real government sites. A 4B model is slow (about 10 seconds a step) and not fully predictable. Required-field detection relies on * marks, required attributes and labels, so unusual forms may slip through. The spending limit adds up every rupee figure on the page, so a page listing several prices could be refused wrongly (amount_regex in the task file fixes that). Irreversible buttons are recognised by the task file or a built-in list of labels, in English only. Real banks add extra steps (3-D Secure, redirects) that we have not covered.

Would it work on a real website?

The design is generic: a new site means a new task JSON (URL, profile, hints, payment button pattern). Expect the model to need tuning, and for real payments the approval screen must stay on (it is the default for non-mock tasks). Dry run exists for exactly this: try the site first with nothing sent, set a spending limit, then resume.

What did the Reddit feedback change?

A commenter on the post asked where this would break on a real government site, whether the approval screen is enough, and suggested a spending limit so the agent cannot pay more than Rs. X without a second approval. In response we built three things: (1) a hard spending limit read from the page (stricter than asked: over the limit it stops, with no way to click through); (2) a dry-run mode, so the agent can be tried on a site without paying; (3) approval for any irreversible button, not only payments. Screenshots of the post, the comment and each change working are in the screenshots/ folder. We did not solve the real-site question: it is still tested only on the mock portal.

Why not use the page's HTML instead of screenshots?

Many portals have broken or obfuscated HTML, canvases and CAPTCHAs, but a person can always read the screen. Vision also generalises across sites. We still use the DOM, but only to verify outcomes.

How do you protect privacy and secrets?

Local model, no cloud calls (there is a test that checks this). Secret values are masked as ••• in the trace and UI. .env, run folders and videos are kept out of GitHub. Video recording can be turned off for sensitive demos.

What would you do next?

Speed (smaller screenshots, reuse plans when the page is unchanged, a bigger model only when stuck); notifications to your phone for OTP and approval; testing on more portals and a benchmark across models; a sandbox payment gateway test.

How many tests and what is the coverage?

84 automated tests (42 + 7 new test functions: consequence 19, agent 9, actions 15, vision 6, plus parametrised cases). They cover the validator, the model client, the full loop and every failure scenario. They run without a model. Real-model behaviour is checked by the demo script and benchmark, not by unit tests.

10. Last-minute checklist

- Ollama running and model pulled (`curl localhost:11434/api/tags`); close other heavy apps.
- Warm the model once before presenting so the first step isn't slow.
- Nothing else on ports 5050 and 8000. Start with `./run.sh`.
- Have a second terminal ready with the scripted demo command as a backup.
- Keep the repo link and the Reddit post open in tabs. Know one real comment you acted on.
- Be upfront: mock portal, small model, not 100% predictable. Judges trust honest limits more than a perfect-sounding story.